

Overview of Word2vec

Subject: Word2vec

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{j \neq 0} \log p(w_{t+j} | w_t)$$

1.

BioVectors An extension of word vectors for n-grams in biological sequences (e.g. DNA, RNA, and proteins) for bioinformatics applications has been proposed by Asgari and Mofrad. Named bio-vectors (BioVec) to refer to biological sequences in general with protein-vectors (ProtVec) for proteins (amino-acid sequences) and gene-vectors (GeneVec) for gene sequences, this representation can be widely used in applications of machine learning in proteomics and genomics. The results suggest that BioVectors can characterize biological sequences in terms of biochemical and biophysical interpretations of the underlying patterns. A similar variant, dna2vec, has shown that there is correlation between Needleman–Wunsch similarity score and cosine similarity of dna2vec word vectors.

2.

Assessing the quality of a model Mikolov et al. (2013) developed an approach to assessing the quality of a word2vec model which draws on the semantic and syntactic patterns discussed above. They developed a set of 8,869 semantic relations and 10,675 syntactic relations which they use as a benchmark to test the accuracy of a model. When assessing the quality of a vector model, a user may draw on this accuracy test which is implemented in word2vec, or develop their own test set which is meaningful to the corpora which make up the model. This approach offers a more challenging test than simply arguing that the words most similar to a given test word are intuitively plausible.

3.

doc2vec doc2vec, generates distributed representations of variable-length pieces of texts, such as sentences, paragraphs, or entire documents. doc2vec has been implemented in the C, Python and Java/Scala tools (see below), with the Java and Python versions also supporting inference of document embeddings on new, unseen documents. doc2vec estimates the distributed representations of documents much like how word2vec estimates representations of words: doc2vec utilizes either of two model architectures, both of which are allegories to the architectures used in word2vec. The first, Distributed Memory Model of Paragraph Vectors (PV-DM), is identical to CBOW other than it also provides a unique document identifier as a piece of additional context. The second architecture, Distributed Bag of Words version of Paragraph Vector (PV-DBOW), is identical to the skip-gram model except that it attempts to predict the window of surrounding context words from the paragraph identifier instead of the current word. doc2vec also has the ability to capture the semantic 'meanings' for additional pieces of 'context' around words; doc2vec can estimate the semantic embeddings for speakers or speaker attributes, groups, and periods of time. For example, doc2vec has been used to estimate the political positions of political parties in various Congresses and Parliaments in the U.S.

and U.K., respectively, and various governmental institutions.

4.

Mathematical details This section is based on expositions. A corpus is a sequence of words. Both CBOW and skip-gram are methods to learn one vector per word appearing in the corpus. Let V ("vocabulary") be the set of all words appearing in the corpus C . Our goal is to learn one vector $\mathbf{v}_w \in \mathbb{R}^d$ for each word $w \in V$. The idea of skip-gram is that the vector of a word should be close to the vector of each of its neighbors. The idea of CBOW is that the vector-sum of a word's neighbors should be close to the vector of the word.

5.

Analysis The reasons for successful word embedding learning in the word2vec framework are poorly understood. Goldberg and Levy point out that the word2vec objective function causes words that occur in similar contexts to have similar embeddings (as measured by cosine similarity) and note that this is in line with J. R. Firth's distributional hypothesis. However, they note that this explanation is "very hand-wavy" and argue that a more formal explanation would be preferable. Levy et al. (2015) show that much of the superior performance of word2vec or similar embeddings in downstream tasks is not a result of the models per se, but of the choice of specific hyperparameters. Transferring these hyperparameters to more 'traditional' approaches yields similar performances in downstream tasks. Arora et al. (2016) explain word2vec and related algorithms as performing inference for a simple generative model for text, which involves a random walk generation process based upon loglinear topic model. They use this to explain some properties of word embeddings, including their use to solve analogies.

6.

History During the 1980s, there were some early attempts at using neural networks to represent words and concepts as vectors. In 2010, Tomáš Mikolov (then at Brno University of Technology) with co-authors applied a simple recurrent neural network with a single hidden layer to language modelling. Word2vec was created, patented, and published in 2013 by a team of researchers led by Mikolov at Google over two papers. The original paper was rejected by reviewers for ICLR conference 2013. It also took months for the code to be approved for open-sourcing. Other researchers helped analyse and explain the algorithm. Embedding vectors created using the Word2vec algorithm have some advantages compared to earlier algorithms such as those using n-grams and latent semantic analysis. GloVe was developed by a team at Stanford specifically as a competitor, and the original paper noted multiple improvements of GloVe over word2vec. Mikolov argued that the comparison was unfair as GloVe was trained on more data, and that the fastText project showed that word2vec is superior when trained on the same data. As of 2022, the straight Word2vec approach was described as "dated". Transformer-based models, such as ELMo and BERT, which add multiple neural-network attention layers on top of a word embedding model similar to Word2vec, have come to be regarded as the state of the art in natural language processing.

7.

The idea of skip-gram is to represent each word with a vector, such that it is possible to predict the vectors of its neighbors using the vector of a word. The architecture is still linear-linear-softmax, the

same as CBOW, but the input and the output are switched. Specifically, for each word w_i in the corpus, the one-hot encoding of the word is used as the input to the neural network. The output of the neural network is a probability distribution over the dictionary, representing a prediction of individual words in the neighborhood of w_i . The objective of training is to maximize $\sum_i \sum_{j \in N} \ln \Pr(w_{j+i} | w_i)$. In full formula, the loss function is $-\sum_i \sum_{j \in N} \ln \frac{e^{v_{w_{j+i}} \cdot v_{w_i}}}{\sum_{w'} e^{v_{w'} \cdot v_{w_i}}}$. Same as CBOW, once such a system is trained, we have two trained matrices V, V' . Either the column vectors of V or the row vectors of V' can serve as the dictionary. It is also possible to simply define $V' = V^{\top}$, in which case there would no longer be a choice. Essentially, skip-gram and CBOW are exactly the same in architecture. They only differ in the objective function during training.